

Applied Machine Learning

Logistic regression



UNIVERSITY OF
GOTHENBURG

CHALMERS

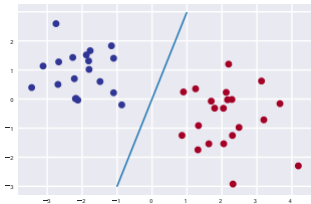
Linear classifier

the “confidence” isn’t
directly interpretable

linear classifiers select the outputs based on a scoring function:

$$\text{score} = \mathbf{w} \cdot \mathbf{x}$$

then predict the positive class if
the score is positive, and vice
versa



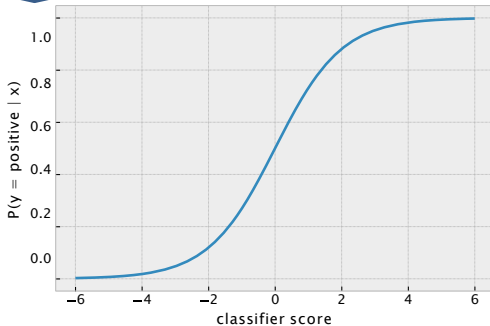
Can we create a model where the output
can be interpreted as a probability?

the logistic regression model

logistic regression is a method to train a linear classifier that gives a probabilistic output

the logistic / sigmoid function [Verhulst, 1845]

The range of the curve is the "probability range" (0,1)



the logistic regression model

- use a **logistic** or **sigmoid** function:

$$P(\text{positive output}|\mathbf{x}) = \sigma(\text{score}) = \frac{1}{1 + e^{-\text{score}}}$$

- this is formally a probability: always between 0 and 1, sum of probabilities of possible outcomes = 1

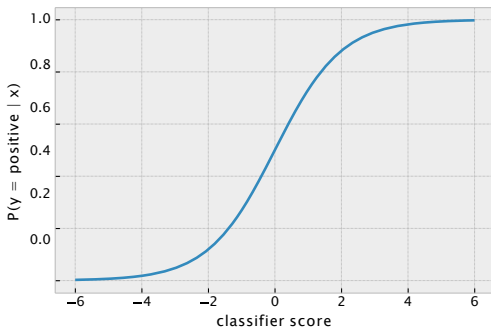
interpreting the linear classifier's score

- the output score $w \cdot x$ can now be interpreted as the log odds in favor of the positive outcome
- odds: how much more likely is the positive outcome than the negative outcome?

$$\text{odds} = \frac{p}{1-p}$$

note the symmetry!

$$\begin{aligned} P(\text{negative output} | \mathbf{x}) &= 1 - \sigma(\text{score}) \\ &= \sigma(-\text{score}) \end{aligned}$$



making it a bit more compact

- if we code the positive class as +1 and the negative class as -1, then we can write the probability a bit more neatly:

$$P(y|\mathbf{x}) = \sigma(y \cdot \text{score})$$

recall: the maximum likelihood principle

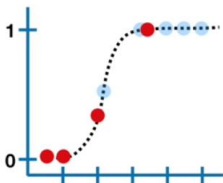
- in a probabilistic model, we can train the model by selecting parameters that assign a high probability to the data
- in our case, the parameters are the weight vector w
- adjust w so that each output label gets a high probability

the likelihood function

- formally, the “probability of the data” is defined by the **likelihood function**
- this is the product of the probabilities of all m individual training instances:

$$L(\mathbf{w}) = P(y_1 | \mathbf{x}_1) \cdot \dots \cdot P(y_m | \mathbf{x}_m)$$

likelihood of data given the squiggle = $0.49 \times 0.9 \times 0.91 \times 0.91 \times 0.92 \times$
 $(1 - 0.9) \times (1 - 0.3) \times (1 - 0.01) \times (1 - 0.01)$



Likelihood of seeing the
whole dataset.

the likelihood function

- formally, the “probability of the data” is defined by the **likelihood function**
- this is the product of the probabilities of all m individual training instances:

$$L(\mathbf{w}) = P(y_1 | \mathbf{x}_1) \cdot \dots \cdot P(y_m | \mathbf{x}_m)$$

- in our case, this means

$$L(\mathbf{w}) = \frac{1}{1 + e^{-y_1 \cdot (\mathbf{w} \cdot \mathbf{x}_1)}} \cdot \dots \cdot \frac{1}{1 + e^{-y_m \cdot (\mathbf{w} \cdot \mathbf{x}_m)}}$$

rewriting a bit...

- we rewrite the previous formula

$$L(\mathbf{w}) = \frac{1}{1 + e^{-y_1 \cdot (\mathbf{w} \cdot \mathbf{x}_1)}} \cdots \frac{1}{1 + e^{-y_m \cdot (\mathbf{w} \cdot \mathbf{x}_m)}}$$

as

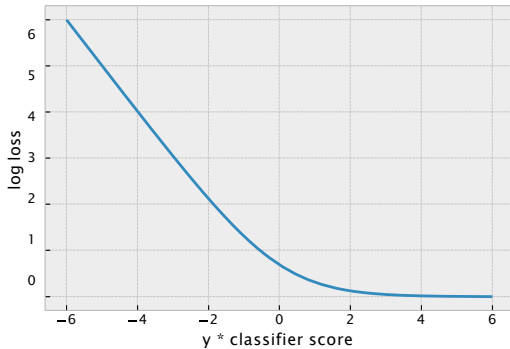
$$-\log L(\mathbf{w}) = \text{Loss}(\mathbf{w}, \mathbf{x}_1, y_1) + \dots + \text{Loss}(\mathbf{w}, \mathbf{x}_m, y_m)$$

where

$$\text{Loss}(\mathbf{w}, \mathbf{x}, y) = \log(1 + \exp(-y \cdot (\mathbf{w} \cdot \mathbf{x})))$$

is called the **log loss** or **binary cross-entropy loss**

plot of the log loss



recall: the fundamental tradeoff in machine learning

- **goodness of fit**: the learned classifier should be able to correctly classify the examples in the training data
- **regularization**: the classifier should be simple
- but so far in our LR description, we've just taken care of the first part!

$$-\log L(\mathbf{w}) = \text{Loss}(\mathbf{w}, \mathbf{x}_1, y_1) + \dots + \text{Loss}(\mathbf{w}, \mathbf{x}_m, y_m)$$

regularization in logistic regression models

- just like we saw for linear regression models (Ridge and Lasso),
- we can add a regularizer that **keeps the weights small**
- most commonly, the **L_2 regularizer**:

$$\|\mathbf{w}\|^2 = w_1 \cdot w_1 + \dots + w_n \cdot w_n = \mathbf{w} \cdot \mathbf{w}$$

- ...or an **L_1 regularizer**:

$$\|\mathbf{w}\|_1 = |w_1| + \dots + |w_n|$$

which will do some feature selection

L2 regularizer

- control the weights by keeping the weights small, but not zero
- works well in case we have less features
- works well in case the features have high correlation
 - Example: Imagine a linear model with two strongly correlated features: these two features are nearly identical copies of one another, but one feature contains a small amount of random noise. By using L2, both features will have roughly equal, moderate weights.
- more useful for feature interpretation: can be used to estimate the significance of predictive features and based on that it can penalize the insignificant ones.

L1 regularizer

- Works well in case we have more features (as it will reduce the weights of non-/less relevant features to zero).
- useful for feature selection

combining the pieces

- we combine the loss and the regularizer:

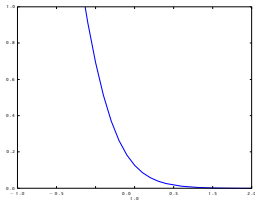
$$\frac{1}{N} \cdot \sum_{i=1}^N \text{Loss}(\mathbf{w}, \mathbf{x}_i, y_i) + \frac{\lambda}{2} \cdot \|\mathbf{w}\|^2$$

- in this formula, λ is a “tweaking” parameter that controls the tradeoff between loss and regularization
- note: in some formulations (including scikit-learn), there is a parameter C instead of the λ that is put before the loss

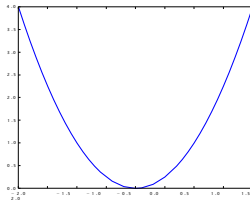
$$\frac{C}{N} \cdot \sum_{i=1}^N \text{Loss}(\mathbf{w}, \mathbf{x}_i, y_i) + \frac{1}{2} \|\mathbf{w}\|^2$$

LR's objective function is convex!

$$\frac{1}{N} \cdot \sum_{i=1}^N \text{Loss}(\mathbf{w}, \mathbf{x}_i, y_i) + \frac{\lambda}{2} \cdot \|\mathbf{w}\|^2$$



+

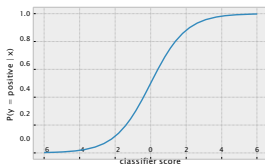


Summay

logistic regression is a common **binary classifier**

it is a **linear classifier**

nice feature: **output probabilities**



we train it by **maximizing the likelihood / minimizing log loss**

Example: LR